

ECE4132 assignment 4: mitigating the effects of integrator windup and discretisation delay

Prepared by: Dr. James Saunderson

Aims

The aims of this assignment are:

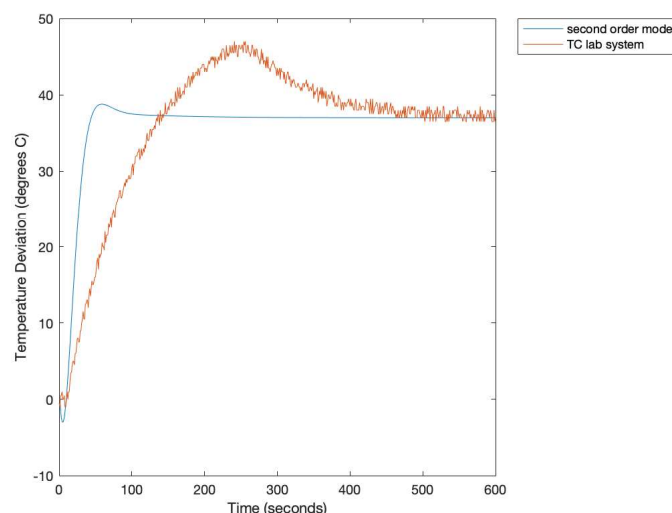
1. To observe the effects of actuator saturation for PI control of the TC lab system.
2. To implement an integrator anti-windup mechanism to mitigate the effect of integrator windup for PI control of the TC lab system.
3. To discretise a PI controller for a second order model of the TC lab system, to obtain a discrete-time controller, and evaluate its performance when the sampling period is long.
4. To improve the design by compensating for the delay introduced by the sample-and-hold.

References and resources

- ‘Getting started with Simulink’ videos from Mathworks: <https://au.mathworks.com/videos/series/getting-started-with-simulink.html>
- K. J. Åström and R. M. Murray *Feedback systems: an introduction for scientists and engineers* Princeton University Press
- G. F. Franklin, D. Powell and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 6th Edition, Pearson and Prentice Hall, 2010.

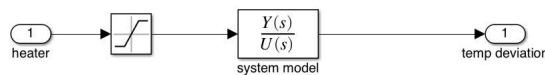
1 Actuator saturation

This assignment picks up where assignment 3 left off. In assignment 3 you investigated PI controllers for a second order model of the TC lab system. At the end of the assignment you saw that there was a significant discrepancy between the response of the closed loop system for the second order system model, and the response of the closed loop system for the actual TC lab system.



One possible reason for this difference is because the heater inputs to the TC lab system saturate—they can only take values between 0 and 100—whereas your second-order model did not involve any saturation. In this section of the assignment, you will first add saturation to your system model and verify that closed loop system using the second order model of the plant with saturation behaves in a similar way to the closed loop system with the TC lab as the plant. You will then implement an integrator windup mitigation method (called back calculation) and see that it improves some of the problems caused by actuator saturation.

- Download the files provided, and open both the MATLAB script `setupModelParametersSat.m` and the Simulink model `TCLabControlSat.slx` (or the alternative model `TCLabControlSat_R2016a.slx` if you are using an older version of Simulink).
- In the Simulink model, add a ‘saturation’ block inside the *SecondOrderModel* subsystem, between the subsystem input and the transfer function block. Set the upper saturation limit to 100 and the lower saturation limit to 0. The *SecondOrderModel* subsystem should now look like:



- Run the script `setupModelParametersSat.m` and then run the Simulink model `TCLabControlSat.slx`.
- Plot the response of the model (i.e., based on the variables `DoutModel.Data` and `DoutModel.Time`) and also plot the response of the TCLab system (based on the variables `Dout.Data` and `Dout.Time`). The plots should look somewhat similar.

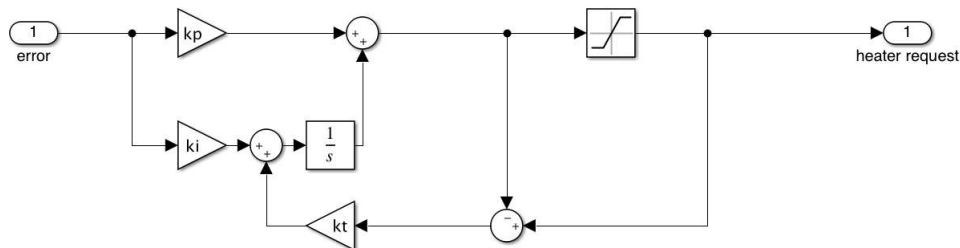
Q1 What is the steady-state value of the output of the *Second Order Model* subsystem? (Your answer should be a whole number.)

- Use `stepinfo` to find step response parameters of the variable `DoutModel.Data`. Make sure that
 - You compute the percentage overshoot and settling time with respect to your answer to Q1. (The default is to use the signal value at the last time-step.)
 - You compute the **5% settling time**. (The default is the 2% settling time.)

To make these modifications, read the documentation at `help stepinfo`.

Q2 What are the percentage overshoot, 10%-90% rise time, and 5% settling time of the output of the second order model with saturation? Give your answers correct to two significant figures.

- In the *model Controller* block, add an integrator anti-windup feedback loop to the controller following the scheme in the video/slides titled “PID control—integrator windup” in the week 6 lecture material. In your parameter-setting script, set the new gain as $k_t = 2k_i/k_p$. The new *model Controller* block should look as follows:



- Once you have finished updating the *model Controller*, make exactly the same changes to the *TCLab controller* subsystem.
- Run the script `setupModelParametersSat.m` and then run the Simulink model `TCLabControlSat.slx`.

- Plot the response of the model (i.e., based on the variables `DoutModel.Data` and `DoutModel.Time`) and also plot the response of the TClab system (based on the variables `Dout.Data` and `Dout.Time`). The plots should look somewhat similar, but quite different from the plots when there was actuator saturation but no back calculation.
- Use `stepinfo` to find step response parameters of the variable `Dout.Data` (the response of the TClab system). As before, ensure that the percentage overshoot and settling time are computed with respect to the known steady-state value of the signal (i.e, your answer to Q1), and that you compute the 5% settling time.

Q3 What are the percentage overshoot, 10%-90% rise time, and 5% settling time of the output of TClab model using the controller with back-calculation? Give your answers correct to two significant figures.

Q4 The integrator anti-windup mechanism you have implemented:

- (reduces/increases/doesn't significantly change) the rise time
- (reduces/increases/doesn't significantly change) the 5% settling time
- (reduces/increases/doesn't significantly change) the percentage overshoot

when compared to the system response with saturation but without the anti-windup mechanism.

2 Basic discrete-time controller implementation

In this section you will aim to regulate the temperature at 30 degrees using a discrete-time controller. (We are aiming to regulate the temperature at a lower value so that actuator saturation does not play as significant a role.) The basic idea of emulation design, is to design a continuous-time controller, and then replace it with a discrete-time approximation. Throughout, we will be using a controller that is obtained by discretising a PI controller using the trapezoidal rule.

Q5 Consider a PI controller with transfer function of the form

$$C(s) = k_p + \frac{k_i}{s}.$$

If this system is approximated via a discrete-time system using the trapezoidal rule (with sampling period T), find the transfer function $C_d(z)$ of the discrete-time system.

- Open the Simulink model `TCLabControlDT.slx` and the MATLAB script `setupModelParametersDT.m`. Inspect both and make sure you understand what they are doing.
- Complete the line of `setupModelParametersDT.m` that computes the transfer function `Cd` of the discrete-time controller (based on the given sampling period of $T = 10$ and the given controller parameters `ki = 0.1` and `kp = 8.75`).
- Run the script `setupModelParametersDT.m` and then run the Simulink model `TCLabControlDT.slx`.
- Plot the output of the system with the continuous-time controller (from the variable `DoutCT`) and the output of the system with the discrete-time controller (from the variable `DoutDT`).
- Compute the percentage overshoot, the 10%-90% rise time, and the 2% settling time of the system with the continuous-time controller (via the variable `DoutCT`). Use `stepinfo`, and ensure you explicitly set the correct steady-state value of the signal.
- Compute the percentage overshoot and the 2% settling time of the system with the discrete-time controller (via the variable `DoutDT`). Use `stepinfo`, and ensure you explicitly set the correct steady-state value of the signal.

Q6 Given that we are choosing a sampling period of 10 seconds, approximately how many samples are there in the 10%-90% rise time of the closed loop continuous-time system? (Round your answer to the nearest whole number.)

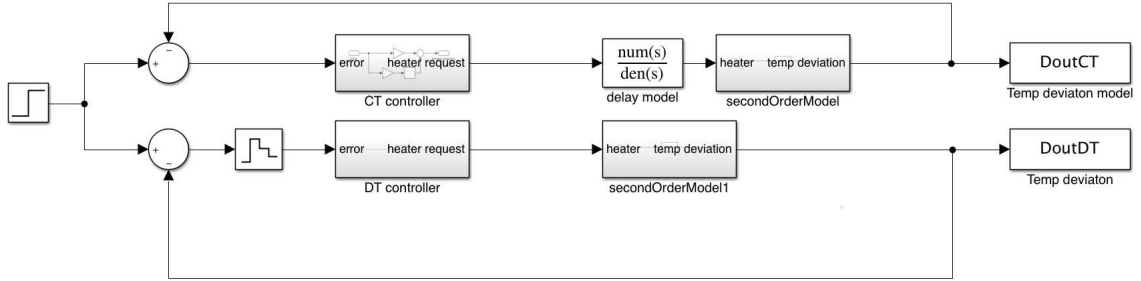
Q7 What is the percentage overshoot and the 2% settling time of the system with a discrete-time controller. (Give your answer to two significant figures.)

You have probably noticed that this is a much lower sampling rate than the recommended 6–10 samples in the closed loop rise time. We are making this choice so that it emphasises the effects of a slow sampling rate, and makes the delay compensation techniques in the next section have a more noticeable effect.

3 Delay compensation

One of the issues with using discrete-time controllers is that the zero-order hold operation introduces a delay into the system of approximately $T/2$, where T is the sampling period. This is one of the reasons that using a long sampling period (as in the previous section) caused significant degradation in performance. In this section, we will explore the effect of explicitly modifying the emulation design process to model the $T/2$ -second average delay introduced by the zero-order-hold.

- In `TClabControlDT.slx` connect a transfer function block in series between the *CT controller* and the *secondOrderModel* as shown below.



- Set the parameters of the new transfer function so that it models a rational approximation of a delay by $T/2$, i.e., the transfer function

$$\frac{1 - \frac{sT}{4}}{1 + \frac{sT}{4}} \approx e^{-sT/2}.$$

- Re-run the Simulink model `TClabControlDT.slx`.
- Plot the output of the system with the continuous-time controller (from the variable `DoutCT`) and the output of the system with the discrete-time controller (from the variable `DoutDT`). These should now be quite similar.

In the previous section, we saw a significant difference between the system behaviour with the CT controller and the DT controller (when the sampling period was large). What you have just done indicates that much of this difference can be explained by the delay introduced by sampling and reconstruction.

Emulation design with delay compensation: This suggests a better way to do emulation design. If we are doing emulation design (with a fairly large sampling period), we should design our continuous-time controller $C(s)$ not for the original plant $P(s)$ (our second order model of the TC lab system), but for the modified plant

$$\tilde{P}(s) = \left(\frac{1 - \frac{sT}{4}}{1 + \frac{sT}{4}} \right) P(s) \approx e^{-sT/2} P(s).$$

The process is:

Modified CT design: Design a continuous-time controller $C(s)$ to achieve desired closed loop behaviour when connected in feedback with the modified plant $\tilde{P}(s) = \frac{1 - Ts/4}{1 + Ts/4} P(s)$.

Discretisation: Discretise that controller with sampling period T to obtain a discrete-time controller $C_d(z)$. Then connecting $C_d(z)$ in feedback with the original plant $P(s)$ should (approximately) achieve the desired closed loop behaviour.

We will see this process in action, next.

- In `setupModelParametersDT.m`, update the controller parameters to `ki = 0.06` and `kp = 6.5` and ensure that the resulting discrete-time approximation `Cd` is also updated accordingly. These parameters are chosen so that the controller $C(s) = k_i/s + k_p$, when interconnected with the delay-compensated continuous time plant $\tilde{P}(s)$, gives desired closed loop system performance.
- Run the Simulink simulation `TllabControlDT.slx`.
- Plot the output of the system with the continuous-time controller and plant $\tilde{P}(s)$ (from the variable `DoutCT`) and the output of the system with the discrete-time controller and the plant $P(s)$ (from the variable `DoutDT`).
- Compute the percentage overshoot and the 2% settling time of the system with the discrete-time controller (via the variable `DoutDT`). Use `stepinfo`, and ensure you explicitly set the correct steady-state value of the signal.

Q8 What is the percentage overshoot and the 2% settling time of the system with the discrete-time controller and controller design incorporating delay compendation. (Give your answer to two significant figures.)